An Exact-Linear-Step (SSCS-style) Sampler for the Active Model

Error Analysis vs. Euler–Maruyama

1 Setup: forward and reverse SDEs

From `compute_mean` in `model_cifar10_sde_DDPM.py`, the forward (noising) process for the active model is the linear system

$$dx = (-kx + \eta)\,dt + \sqrt{2T_p}\,dW_x, \qquad d\eta = -\frac{\eta}{\tau}\,dt + \frac{1}{\tau}\sqrt{2T_a}\,dW_\eta, \tag{1}$$

with $dW_x \perp dW_\eta$. Writing $z = (x, \eta)^\top$, this is $dz = Az\,dt + B\,dW$ with

$$A = \begin{pmatrix} -k & 1 \\ 0 & -1/\tau \end{pmatrix}, \qquad Q \equiv BB^\top = \begin{pmatrix} 2T_p & 0 \\ 0 & 2T_a/\tau^2 \end{pmatrix}. \tag{2}$$

The current sampler (`sampling()`, lines 810–839) discretizes the score-corrected *reverse*-time SDE

$$dx = \Big[(kx - \eta) + 2T_p F_x\Big]dt + \sqrt{2T_p}\,dW_x, \qquad d\eta = \Big[\frac{\eta}{\tau} + \frac{2T_a}{\tau^2}F_\eta\Big]dt + \frac{1}{\tau}\sqrt{2T_a}\,dW_\eta, \tag{3}$$

where (F_x, F_η) is the network's score output, using plain Euler–Maruyama (EM) with fixed step dt.

2 Splitting: linear part L vs. score part N

The reverse drift in (3) splits additively into a *linear* part and a *nonlinear* (score) part:

$$\underbrace{\begin{pmatrix} kx - \eta \\ \eta/\tau \end{pmatrix}}_{\tilde{A}z,\ \ \tilde{A}=-A} + \underbrace{\begin{pmatrix} 2T_p F_x \\ (2T_a/\tau^2)F_\eta \end{pmatrix}}_{\mathrm{Force}(z)}. \tag{4}$$

The key fact: the $\tilde{A}z$ part, together with its own noise, is an *exactly solvable* linear SDE — the same OU structure as the forward process in (1), just with the sign of the drift flipped ($\tilde{A} = -A$). We integrate that part in closed form and reserve Euler only for the genuinely nonlinear score term.

2.1 Exact solution of L: $dz = \tilde{A}z\,dt + B\,dW$

Diagonalizing $\tilde{A} = \begin{pmatrix} k & -1 \\ 0 & 1/\tau \end{pmatrix}$ (eigenvalues $k,\ 1/\tau$, assumed distinct — same non-degeneracy already assumed by `compute_mean`) gives

$$e^{\tilde{A}u} = \begin{pmatrix} e_1(u) & \dfrac{e_2(u) - e_1(u)}{\Delta} \\ 0 & e_2(u) \end{pmatrix}, \qquad e_1(u) = e^{ku}, \quad e_2(u) = e^{u/\tau}, \quad \Delta = k - \frac{1}{\tau}. \tag{5}$$

Mean. Over a step of length s, starting from (x_0, η_0):

$$\bar{x}(s) = e_1(s)\,x_0 + \frac{e_2(s) - e_1(s)}{\Delta}\,\eta_0, \qquad \bar{\eta}(s) = e_2(s)\,\eta_0. \tag{6}$$

(This is algebraically identical to `compute_mean`$(x_0, \eta_0, t{=}{-}s)$ — the reverse linear flow is the forward linear flow run at negative time, since $\tilde{A} = -A$.)

Covariance. The noise covariance is *not* simply the forward covariance at $t = -s$, because Q (fixed) enters the Lyapunov integral $\Sigma(s) = \int_0^s e^{\tilde{A}u} Q\, e^{\tilde{A}^\top u}\,du$ with the flipped-sign propagator. Carrying out the integral (with $q_1 = 2T_p$, $q_2 = 2T_a/\tau^2$):

$$\Sigma_{\eta\eta}(s) = \frac{T_a}{\tau}\big(e_2(s)^2 - 1\big), \tag{7}$$

$$\Sigma_{x\eta}(s) = \frac{q_2}{\Delta}\left[\frac{\tau}{2}\big(e_2(s)^2 - 1\big) - \frac{e_1(s)e_2(s) - 1}{k + 1/\tau}\right], \tag{8}$$

$$\Sigma_{xx}(s) = \frac{q_1\big(e_1(s)^2 - 1\big)}{2k} + \frac{q_2}{\Delta^2}\left[\frac{\tau}{2}\big(e_2(s)^2 - 1\big) - \frac{2\big(e_1(s)e_2(s) - 1\big)}{k + 1/\tau} + \frac{e_1(s)^2 - 1}{2k}\right]. \tag{9}$$

(Sanity check: as $s \to 0$, $\Sigma_{\eta\eta} \to q_2 s$, $\Sigma_{xx} \to q_1 s$, $\Sigma_{x\eta} = O(s^2)$ — matches independent-noise-source diffusion at leading order, since cross-correlation can only appear via drift coupling, which is higher order in s.)

Because T_p, T_a, k, τ are constants (no time-dependent noise schedule), the 2×2 matrix $[\bar{x}, \bar{\eta}] = M(s)[x_0, \eta_0]^\top$ and $\Sigma(s)$ depend *only on the step size* s, not on absolute time — so for a fixed step schedule, $M(dt/2)$ and $\Sigma(dt/2)$ can be precomputed once, outside the sampling loop.

3 The Strang-split (SSCS-style) update

For each outer step dt, with half-step $s = dt/2$:

1. **Analytic half-step** (exact, no network call): sample $(x', \eta') \sim \mathcal{N}\big(M(s)\,(x, \eta),\ \Sigma(s)\big)$ via (6)–(9) (Cholesky of $\Sigma(s)$, per channel).

2. **Score Euler step** (one network evaluation, at (x', η'), time $t_{\text{curr}} - dt/2$), deterministic only:

$$x'' = x' + 2T_p F_x\,dt, \qquad \eta'' = \eta' + \frac{2T_a}{\tau^2} F_\eta\,dt. \tag{10}$$

3. **Analytic half-step again**, same formulas as step 1, applied to (x'', η''), giving $(x_{\text{next}}, \eta_{\text{next}})$.

This uses exactly one network evaluation per outer step — the same cost as the current Euler–Maruyama loop.

4 Error analysis: why this beats Euler–Maruyama

4.1 Euler–Maruyama: order and the stiffness problem

EM applied to (3) has the standard orders (weak) $= 1$, (strong) $= 1/2$: global weak error $O(dt)$, global strong error $O(\sqrt{dt})$. But the more important issue here is not the asymptotic order — it is the *size of the constant*, which blows up as $\tau \to 0$.

Consider just the deterministic η-update in isolation: $d\eta/ds = \eta/\tau$. EM approximates the exact one-step propagator $e^{dt/\tau}$ by the linearization $1 + dt/\tau$. The local (one-step) truncation error is

$$e^{dt/\tau} - \Big(1 + \frac{dt}{\tau}\Big) = \frac{1}{2}\Big(\frac{dt}{\tau}\Big)^2 + O\Big(\big(\tfrac{dt}{\tau}\big)^3\Big). \tag{11}$$

This is the usual $O(dt^2)$ local truncation error of a first-order method — *but* the coefficient scales as $1/\tau^2$. For fixed dt, shrinking τ inflates this error rapidly: once $dt/\tau \gtrsim 1$, the linear approximation $1 + dt/\tau$ is no longer even close to $e^{dt/\tau}$, and the scheme is in the stiff/unstable regime (EM's stability threshold for a mode with rate $1/\tau$ requires $dt/\tau \lesssim 2$ just to stay bounded, let alone accurate). This is a direct, quantitative explanation for the tau= 0.25 vs. tau= 0.45 FID gap observed empirically: at fixed $dt = T/N_{\text{steps}}$, the smaller τ pushes dt/τ higher, inflating exactly this term.

4.2 Exact linear step: removes the stiffness error entirely

Because (6)–(9) are the *exact* solution of the linear part for *any* step size s, this error term vanishes identically — not asymptotically, exactly — regardless of how large dt/τ is. There is no discretization bias on the linear/OU part of the dynamics at any step size; the only question is how much of the reverse SDE's drift is linear vs. score-coupled, and the noise covariance is likewise exact (both mean and full covariance are closed-form, not approximated).

4.3 Remaining error: only from the score (splitting) term *(corrected)*

Caveat this section originally missed. The claim below of full second-order accuracy is *not correct as the scheme was described* in an earlier draft, and the fix requires stating things carefully.

True Strang splitting is $e^{h(L+N)} \approx \varphi_L^{h/2} \circ e^{hN} \circ \varphi_L^{h/2}$, which requires the middle factor to be the *exact* flow of N. Our middle step is instead a single frozen-force Euler update $z \mapsto z + hN(z)$ — an $O(h)$ (first-order) approximation of e^{hN}, not the exact flow. Expanding both the scheme and the true solution to $O(h^2)$ (with $z = (x, \eta)$, $L(z) = (kx - \eta, \eta/\tau)$, $N(z) = (2T_p F_x,\ (2T_a/\tau^2)F_\eta)$) shows every cross term between L and N ($L'L$, $L'N$, $N'L$) matches exactly between the two — this is the real benefit of symmetric placement: it cancels the L–N *splitting commutator* error that a naive sequential (Lie–Trotter) scheme would incur.

What does *not* cancel is the term intrinsic to how N alone is integrated:

$$u_3 - z(dt) = -\frac{dt^2}{2} N'(z_0)N(z_0) + O(dt^3), \tag{12}$$

because the true flow has $e^{dtN}(z_0) = z_0 + dt\, N(z_0) + \frac{dt^2}{2} N'(z_0)N(z_0) + O(dt^3)$ while Euler keeps only the first term. No amount of symmetric bracketing repairs an error internal to a single sub-flow's own integrator — only errors arising from *interaction* between L and N.

Concretely, in the original variables,

$$N'(z)N(z) = \begin{pmatrix} 4T_p^2 F_x\, \partial_x F_x + \dfrac{4T_pT_a}{\tau^2} F_\eta\, \partial_\eta F_x \\ \dfrac{4T_pT_a}{\tau^2} F_x\, \partial_x F_\eta + \dfrac{4T_a^2}{\tau^4} F_\eta\, \partial_\eta F_\eta \end{pmatrix}, \tag{13}$$

i.e. Jacobian–vector products of the network with itself. Two things are worth noting about this term: (i) it is genuinely present, so the scheme is **globally first order, not second order**; but (ii) it carries no $1/\tau$ divergence at fixed step size the way EM's linear-term error (11) does — the worst power of τ here is τ^{-4} multiplying a *bounded* network output, not an exponentially/rationally diverging linear term. So the practically dangerous, τ-dependent stiffness error is still fully removed; what remains is a smooth, τ-independent $O(dt^2)$ error from the score network's self-nonlinearity.

4.4 Summary *(corrected)*

	Euler–Maruyama	Exact-linear + symmetric split
Linear $(x, \eta$ OU) part error	$O(dt^2/\tau^2)$ per step, stiff	0 (exact)
L–N splitting/commutator error	$O(dt^2)$ (present, unsandwiched)	0 (cancelled by symmetry)
N self-nonlinearity error	lumped into same term	$O(dt^2)$ per step, τ-independent
Global order	1	1 *(not 2 — corrected)*
Behavior as $\tau \to 0$ at fixed dt	degrades (stiffness)	unaffected
Network evals per step	1	1 (same cost)

4.5 Getting to genuine second order

Two concrete routes, both worth weighing against the current 2-NFE Heun PF-ODE sampler already in the codebase:

- **RK2/midpoint N-step (2 NFEs per step).** Replace the Euler substep with a Heun-style predictor–corrector for N alone (evaluate the score at z', take an Euler predictor, evaluate the score again at the predicted point, average). This recovers the missing $\frac{1}{2}N'N$ term to leading order and restores genuine local error $O(dt^3)$ / global order 2, at double the network cost per step.
- **Score extrapolation (1 NFE per step).** Approximate the missing term using previously computed score values instead of an extra evaluation at the current step — an Adams–Bashforth-style linear multistep exponential integrator, in the spirit of

DPM-Solver [4] and DEIS [5] for diffusion ODEs (both of which are built on exactly this exact-linear-part-plus-extrapolated-nonlinear-part idea). Needs a warm-start for the first step and some extra bookkeeping to cache scores across steps, but keeps the 1-NFE-per-step cost of the current sampler.

5 Final-step correction: an explicit Tweedie estimate at t_{eps}

Both active samplers above stop at $t_{\text{eps}} = 10^{-3}$ (PF-ODE) or $t = dt$ (SDE/SSCS) rather than the untrained $t = 0$, for the same reason documented in `tweedie_denoising_note.tex` for the passive model: the score network never saw $t \approx 0$ during training. Stopping early avoids the out-of-distribution evaluation, but leaves the state sitting on the noisy marginal $p_{t_{\text{eps}}}(x, \eta \mid x_0, \eta_0)$ rather than a clean estimate of x_0 — genuine residual spread, not sampler-injected noise.

5.1 Exact correction

Unlike the passive model, η_0 is also random (drawn from its own stationary prior, independent of the data x_0), so the relevant identity is a joint (not scalar) Tweedie/Miyasawa estimate. Writing $w = (x_0, \eta_0)^\top$, $F = \begin{pmatrix} a & b \\ 0 & c \end{pmatrix}$ (the mean-map from §2.1), and $\Sigma_{\text{cond}} = M(t_{\text{eps}})$ (the network's actual denoising-score-matching target covariance, from `compute_covariance`), the general linear-Gaussian Tweedie identity $\nabla_z \log p_t(z) = -\Sigma_{\text{cond}}^{-1}(z - F\bar{w})$ gives, after inverting F and keeping only the x_0 component:

$$\hat{x}_0 = \frac{1}{a}\left[(x_t + M_{11}F_x + M_{12}F_\eta) - \frac{b}{c}\,(\eta_t + M_{12}F_x + M_{22}F_\eta)\right]. \tag{14}$$

This is exact given exact scores, and was verified numerically against direct multivariate Gaussian conditioning (synthetic Gaussian x_0 prior, so a ground-truth posterior mean is computable independently) — the formula reproduced the ground truth to full floating-point precision.

5.2 Explicit form when $\tau \gg t_{\text{eps}}$ and $T_p \ll 1$

Both hold for every configuration used in this codebase ($T_p = 10^{-3}$, $\tau \in \{0.25, 0.45\}$, $t_{\text{eps}} = 10^{-3}$). Expanding (14) to leading order in t_{eps} ($a \approx 1$, $c \approx 1$, $b \approx t_{\text{eps}}$, $M_{12} = O(t_{\text{eps}}^2)$, $M_{11} \approx 2T_p t_{\text{eps}}$):

$$\hat{x}_0 \approx x_t - t_{\text{eps}}\,\eta_t + 2T_p\,t_{\text{eps}}\,F_x + O(t_{\text{eps}}^2). \tag{15}$$

Checked numerically against (14) at the true parameter values: relative error $\sim 0.1\%$, consistent with the $O(t_{\text{eps}}^2)$ truncation. The $2T_p t_{\text{eps}} F_x$ term costs nothing extra to keep (the network is already evaluated once more at t_{eps} for this correction, same as the passive model's Tweedie step), so (15) — not a further-truncated $\hat{x}_0 \approx x_t - t_{\text{eps}}\eta_t$ — is what's implemented in `model_cifar10_sde_DDPM.py`'s `_active_tweedie_correction`, called from the PF-ODE and SDE branches of `sampling()` and from `sampling_sscs()`, gated by a `tweedie=True` flag matching the passive model's convention.

6 Connection to CLD-SGM

This is the same construction as the Symmetric Splitting CLD Sampler (SSCS) in Dockhorn et al. (ICLR 2022): their auxiliary velocity variable v plays the role of our η, their damping/friction parameters play the role of our k, τ, and their `analytical_dynamics` half-steps are the exact OU propagation derived above, sandwiching a `euler_score_dynamics` step that only touches the network-dependent force term. The one difference is that CLD's noise schedule $\beta(t)$ is time-dependent, so their closed-form coefficients involve β_{int}; our T_p, T_a, k, τ are constant, which makes our half-step transition matrix and covariance step-size-only (precomputable once). *Note:* Dockhorn et al. justify SSCS by the same stiffness-removal argument used here, not by a claim of formal second-order accuracy for the composed sampler — the order correction in §4.3 applies equally to SSCS itself, not just to this adaptation of it.

References

[1] T. Dockhorn, A. Vahdat, and K. Kreis, *Score-Based Generative Modeling with Critically-Damped Langevin Diffusion*, ICLR 2022.

[2] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, *Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021.

[3] G. Strang, *On the Construction and Comparison of Difference Schemes*, SIAM J. Numer. Anal. 5(3), 1968.

[4] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, *DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps*, NeurIPS 2022.

[5] Q. Zhang and Y. Chen, *Fast Sampling of Diffusion Models with Exponential Integrator*, ICLR 2023.